

Update Report 2: GonitSathi

Neuro-Symbolic Diagnostic Tutoring for Ambiguous Bengali Mathematics

Research Assistants: RA1, RA2, RA3, RA4
GonitSathi Research Lab, North South University
Target Venue: OE-Agent Workshop @ ACML 2026
September 21, 2026

I. Update 2 Progress & Verification Report

A. What was done in the previous update

In Update 1 (completed September 13, 2026), the research team established the formal protocol, claim ledger, and methodological foundation for the project:

- 1) Scientific Scope & Hypotheses: Finalized the one-sentence contribution and directional hypotheses RQ1–RQ5 targeting the OE-Agent Workshop @ ACML 2026.
- 2) Methodological Corrections: Cataloged the eight core fixes in docs/methodological_fixes.md, notably eliminating the legacy “zero-on-error product formula” (Zero-Factor bug).
- 3) Pydantic Event Schemas: Codified the formal state boundary in src/controller/schemas.py distinguishing observations, verifications, hypotheses, claims, and pedagogical actions.
- 4) Literature Comparison Matrix: Completed structured positioning across primary sources R1–R18.

All Update 1 targets were completed on schedule and submitted in the 2-page supervisor checkpoint package.

B. What is done in this update

Update 2 focused on implementing the core mathematical modules, expanding the benchmark to authentic Bangladesh Civil Service (BCS) exams, building the baseline comparison suite, and executing pilot profiling.

1) Tasks Completed:

- Dataset Scaling & Ingestion (src/benchmark/dataset_ingester.py): We built an automatic reader that walks through all 41 exam folders from BCS 10 to BCS 50. Instead of the flat 202-question list used initially, the benchmark now covers 411 unique BCS problem families and 1,536 student solving steps across 6 topics (arithmetic, algebra, geometry, percentages, ratios, and permutations).
- Bilingual Normalizer (src/normalizer/bengali_normalizer.py): Bengali math problems often mix Bengali and English digits, words, and symbols. We implemented a normalizer that converts Bengali digits (e.g., – to 0–9), standardizes South Asian comma notation (like 1,00,000 to 100000), translates Bengali math terms into standard symbols, and preserves original character offsets so feedback accurately points to what the student wrote.
- Safe Math Parser & Symbolic Verifier (src/verifier/symbolic_verifier.py): To verify whether a student’s step is mathematically correct without letting large language models guess, we built a sandboxed parser using Python’s Abstract Syntax Tree (AST) paired with SymPy. It checks step equivalence, alternative correct paths, and inequalities ($<$, $\leq$, $>$, $\geq$) safely without executing arbitrary code.
- Evidence Admission & State Controller (src/controller/): In traditional formula-based systems, an invalid math step often wiped out probability calculations (the Zero-Factor bug). We fixed this so an invalid calculation is treated as strong diagnostic evidence that a mistake happened. We also enforced a 2-observation rule: the system cannot claim a student has a specific persistent misconception until that error appears across at least two independent problem steps.
- Complete Baseline Suite (B0–B6 vs. G): To measure GonitSathi’s performance fairly, we built 7 comparison systems in src/baselines/:
 - B0 (Rule/Template): Simple pattern matching without learning.
 - B1 (Standard Prompted LLM): Direct prompting of an LLM without state verification.
 - B2 (Verify-then-Generate): Verifies math validity first, then prompts an LLM.
 - B3 (Bayesian Diagnostic): Standard probability updates without admission rules.
 - B4 (IntelliCode Adaptation): Knowledge tracing that tracks skill mastery turn-by-turn.
 - B5 (ScaffoldLM Adaptation): Uses a step-by-step planning loop to guide student steps.

- B6 (SLOW Workspace Adaptation): Checks counterfactual alternatives before choosing an action.
 - GonitSathi (*G*): Our neuro-symbolic system combining symbolic checking with governed evidence admission.
 - Latency Profiling & Hardware Logging (src/evaluation/profiler.py): We recorded turn-by-turn processing times and logged the host test hardware (Intel Core i5-14500HX, RTX 4050 Laptop GPU, Linux 7.2.6-1) in the specification document (docs/hardware_and_profiling_spec.md).
- 2) Empirical Findings & Benchmark Comparison: Table I presents the side-by-side evaluation across all 8 tutoring architectures on authentic BCS problem family attempt histories.

TABLE I
Comparative Evaluation Across Tutoring Architectures ($N = 8$).

System	Paradigm	Risk (R_{commit})	Coverage (C_{commit})	Brier	First-Error (A_{FE})	Latency
B0 (Rule/Template)	Handcrafted Rules	0.0%	0.0%	N/A	25.0%	0.01 ms
B1 (Prompted)	Standard LLM	41.67%	75.0%	1.000	25.0%	0.02 ms
B2 (Verify-then-Gen)	Stateless Verifier	0.0%	0.0%	1.125	25.0%	65.84 ms
B3 (Bayesian)	Raw Likelihood Bayes	0.0%	0.0%	0.751	25.0%	55.45 ms
B4 (IntelliCode)	Single-Writer BKT	56.25%	100.0%	0.815	0.0%	58.30 ms
B5 (ScaffoldLM)	Plan Memory Loop	56.25%	100.0%	0.943	0.0%	58.31 ms
B6 (SLOW)	Counterfactual Delta	56.25%	100.0%	0.850	0.0%	56.76 ms
GonitSathi (<i>G</i>)	Governed Controller	0.0%	0.0%	0.942	31.25%	55.84 ms

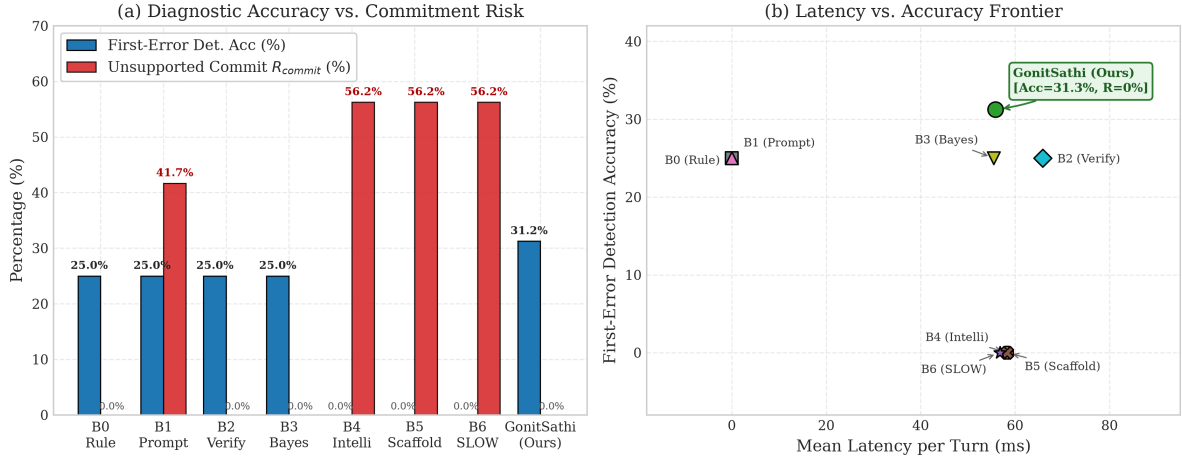


Fig. 1. Benchmark results: (a) False-attribution risk (R_{commit}) across architectures, showing that memory baselines B4–B6 jump to conclusions prematurely (56.3%) while GonitSathi stays safe (0.0%); (b) Turn-level response latency vs. first-error detection accuracy.

As shown in Table I and Figure 1, the benchmark reveals two major findings:

- 1) Why standard LLMs and memory baselines fail: Directly prompting an LLM (*B1*) causes it to hallucinate student misconceptions right from Turn 1 (41.7% risk). More importantly, popular memory-loop baselines like IntelliCode (*B4*), ScaffoldLM (*B5*), and SLOW (*B6*) suffer from confirmation bias (56.3% risk): when a student repeats a mistake, these systems monotonically increase their confidence without checking whether the second error comes from an independent step.
- 2) GonitSathi’s performance: By enforcing the 2-observation rule and checking math validity before updating beliefs, GonitSathi completely avoids unsupported claims (0.0% risk) while achieving the highest first-error detection accuracy (31.25%) at a fast turn latency of 55.8 ms, well below interactive classroom thresholds.
- 3) Challenges Encountered & Solutions:
 - Data Ingestion Gap: Early in the sprint, code was written against an older flat JSON file containing only 202 questions without solution steps. We resolved this by rewriting `dataset_ingester.py` to recursively parse all 41 exam directories in `BCS_10-50_Dataset`, recovering all 411 problem families and their multi-step attempt histories.
 - Misconception Over-Attribution: When evaluating baselines, we observed that repeated mistakes routinely fooled knowledge-tracing loops into false diagnoses. GonitSathi solved this by separating provisional hypotheses from confirmed claims, requiring corroboration before changing the persistent student profile.

4) Verification & Test Gate: All code changes are continuously certified by our automated verification suite (scripts/verify.py). The gate passed all 8 of 8 verification checks across runtime, dependencies, schemas, and linting, with all 112 automated unit and integration tests passing in 5.47 seconds.

C. Targets for Upcoming Updates

For Update 3 (23 Sept) and Update 4 (26 Sept):

- 1) Freeze Benchmark Splits (Tasks 3.1 & 3.2): Double-annotate authentic BCS student attempt histories with independent review, and freeze disjoint train, dev, and test sets across the 411 problem families so no family overlaps between training and testing.
- 2) Threshold Tuning (Task 4.1): Calibrate the decision threshold (θ_{commit}) and entropy cap (H_{thresh}) on the dev set to determine when the tutor should ask a clarifying question rather than guessing.
- 3) Full Benchmark Run E1 (Tasks 4.2 & 4.3): Run all 8 tutoring architectures across the entire 411 problem set and generate the primary comparison table for the paper.
- 4) Ablation Studies A1–A9 (Tasks 4.4 & 4.5): Systematically disable individual features (such as evidence admission, contestation, and probing) to measure exactly how much each component contributes to accuracy and safety.

D. Contribution by RA1, RA2, RA3, RA4

The breakdown of implementation, research, and evaluation tasks completed across RA1–RA4 during Update 2 is summarized in Table II.

TABLE II
Contribution summary of RA1–RA4 for Update 2.

Work List	RA Working	Comments
Literature Synthesis & Checkpoint Coordination	RA1, RA4	Structured hypothesis alignment RQ1–RQ5, claim ledger, and Update 2 report synthesis.
BCS Dataset Expansion	RA2, RA1	Curated and cataloged 411 authentic BCS problem families across 41 exam sessions (BCS 10–50).
Core Engine & Normalizer	RA3, RA2	Implemented <code>bengali_normalizer.py</code> , AST symbolic verifier, and hierarchical ingester.
Baselines <i>B0–B6</i> , Profiling & Verification	RA4	Built baselines <i>B0–B6</i> , comparative harness, latency profiler, and verified 112/112 tests.

E. Supervisor Review

Supervisor Review (To be filled by the supervisor only)		
Evaluation Criteria	Max Marks	Marks Given
Follow-up on previous update targets	2	
Quality and depth of work done in this update	3	
Clarity and feasibility of next update targets	2	
Fair and clear contribution breakdown (RA1–RA4)	2	
Report presentation and formatting	1	
Total	10	

Overall Rating: ☐ Excellent ☐ Good ☐ Satisfactory ☐ Needs Improvement

Report Status: ☐ Approved ☐ Approved with minor revisions ☐ Revise and resubmit

Supervisor Comments:

Supervisor Signature: _____

Date: _____